

INSTITUCIÓN UNIVERSITARIA PASCUAL BRAVO
FACULTAD DE INGENIERÍA • DEPARTAMENTO DE SISTEMAS
BASES DE DATOS I

INFORME TÉCNICO DE AULA RTF4
IMPLEMENTACIÓN Y REFINAMIENTO DEL MODELO
LÓGICO DDL

**Motor Relacional de Gestión Económica, Trazabilidad Contable e Inmutabilidad Transaccional
para Servidores de Rol**

PROYECTO DE AULA:	Economy RP (Trade OS & Engine)
EQUIPO DE TRABAJO:	Grupo 4
DOCENTE ASESOR:	Juan Camilo Palacio Alcaraz
INTEGRANTES:	• Miguel Ángel Cardona Agudelo • Luisa María López Orrego • Jorge Luis Ordoñez Ávila
SEMESTRE / FECHA:	2026-2 • 29 de Septiembre de 2026

RÚBRICA DE EVALUACIÓN OFICIAL RTF4 (100% PONDERADO)	
1. Evolución y Refinamiento del Modelo Lógico (Antes vs. Después)	25% (1.25)
2. Definición y Estructura del Esquema DDL Tabular	25% (1.25)
3. Informe de Mejoras y Proceso de Normalización (1FN, 2FN, 3FN)	20% (1.00)
4. Pruebas Estructurales, Hallazgos y Manejo de Errores de Motor	20% (1.00)
5. Estructura, Formato Institucional y Defensa Técnica	10% (0.50)

TABLA DE CONTENIDO

1. Introducción y Contexto del Proyecto	3
1.1 Descripción del Negocio y Problemática Operativa	3
1.2 Objetivos del Sistema Relacional	3
1.3 Catálogo Oficial de Reglas de Negocio (RN001 - RN012)	3
2. Criterio 1: Evolución y Refinamiento del Modelo Lógico (Antes vs. Después) [25%]	5
2.1 Línea de Tiempo: De la Abstracción Teórica a la Implementación DDL	5
2.2 Diagnóstico de Fallas Críticas en el Modelo Lógico Inicial	5
2.3 Matriz de Refinamiento: Transformación Estructural Antes vs. Después	6
3. Criterio 2: Definición y Estructura del Esquema DDL Tabular [25%]	7
3.1 Diccionario Tabular Estricto de Entidades y Restricciones	7
3.2 Justificación Técnica de Tipos de Datos y Dominios de Integridad	10
4. Criterio 3: Informe de Mejoras y Proceso de Normalización [20%]	11
4.1 Justificación Formal de 1FN: Atomicidad y Llaves Primarias	11
4.2 Justificación Formal de 2FN: Eliminación de Dependencias Parciales	11
4.3 Justificación Formal de 3FN: Supresión de Dependencias Transitivas	12
4.4 Matriz Resumen de Cumplimiento de Formas Normales	12
5. Criterio 4: Pruebas Estructurales, Hallazgos y Manejo de Errores [20%]	13
5.1 Metodología de Validación de Integridad en el Motor SGBD	13
5.2 Bitácora de Casos de Prueba, Errores de Motor y Mitigación DDL	13
6. Criterio 5: Estructura, Formato y Defensa Técnica del Modelo [10%]	15
6.1 Defensa Técnica: Por qué el Modelo Permite o Restringe el Flujo de Datos	15
6.2 Garantía de Propiedades ACID y Consistencia Relacional	15
6.3 Certificación de Cumplimiento y Firmas del Equipo	15

1. INTRODUCCIÓN Y CONTEXTO DEL PROYECTO

1.1 Descripción del Negocio y Problemática Operativa

Los servidores privados de juego de rol (Roleplay - RP) basados en entornos multijugador masivos (FiveM, RedM, RAGE MP) sostienen una economía virtual dinámica donde los jugadores desempeñan actividades laborales simuladas y comercializan activos de alto valor (vehículos, residencias, licencias comerciales). Tradicionalmente, esta economía ha sido gestionada mediante scripts independientes y archivos de texto plano desvinculados, carentes de un motor relacional centralizado con control de integridad referencial. Esta deficiencia genera fallas críticas como duplicación ilegal de activos (*item duplication*), saldos negativos por carreras de concurrencia, transferencias bilaterales inconclusas (cuando una parte entrega el bien pero no recibe el pago por fallas de red), hiperinflación descontrolada por emisión salarial sin respaldo, y una altísima carga operativa de soporte técnico manual incapaz de auditar incidentes de forma fehaciente.

Economy RP es una solución de ingeniería de bases de datos diseñada bajo el motor relacional MySQL (InnoDB), cuyo propósito es centralizar la gestión de jugadores, cuentas financieras, inventarios de bienes, jornadas laborales y transacciones comerciales en un esquema estrictamente normalizado (3FN) con inmutabilidad contable por partida doble.

1.2 Objetivos del Sistema Relacional

- **Garantía de Atomicidad e Integridad Transaccional:** Asegurar que ningún intercambio P2P quede truncado a la mitad mediante transacciones ACID y bloqueos pesimistas.
- **Prevención de Duplicación y Fraude:** Restringir la posesión de cada ítem a un único propietario legítimo mediante llaves foráneas unívocas y validación de gravámenes.
- **Trazabilidad Contable e Inmutabilidad:** Registrar cada movimiento financiero con partida doble y prohibición estricta de borrado físico o modificación en el libro mayor.
- **Mecanismo Anti-Inflacionario:** Incorporar un drenaje fiscal automático por comisión en transacciones hacia la tesorería del sistema.
- **Automatización de Políticas Laborales:** Validar en el motor de base de datos los periodos de enfriamiento (cooldown) y el cálculo inmutable de salarios.

1.3 Catálogo Oficial de Reglas de Negocio (RN001 - RN012)

A continuación se presenta el marco normativo del negocio formalizado en las etapas conceptuales y codificado en el esquema DDL:

Cód.	Nombre de la Regla	Descripción Funcional y Lógica	Impacto DDL / DB
RN001	Registro Único Laboral	Al completar un empleo, el sistema calcula y acredita automáticamente la ganancia: $Ganancia = Tarifa_base \times Horas$.	Tabla T_Jornada_Laboral, trigger de liquidación y SP.
RN002	Propiedad Única y Exclusiva	Un bien físico (vehículo con placa, propiedad) solo puede pertenecer a un único personaje a la vez.	FK T_Item.id_jugador atómica (1 dueño por ítem).
RN003	Inmutabilidad del Historial	Toda transacción registrada es inmutable. Prohibido editar o eliminar registros de auditoría.	Triggers BEFORE UPDATE/DELETE con SIGNAL 45000.
RN004	Restricción de Transferencia por Deuda	Un bien no puede transferirse si posee deuda activa, embargo o cuota pendiente.	Columna tiene_deuda BOOLEAN y validación en SP/CHECK.
RN005	Trazabilidad de Creación de Dinero	Todo dinero inyectado (salarios, bonos) debe originarse desde la cuenta oficial 'SISTEMA'.	Trigger trg_detalle_origen_salario y cuenta tipo SISTEMA.
RN006	Comisión por Transferencia	Toda transacción P2P descuenta una tasa fija configurable acreditada a la tesorería del sistema.	Campo T_Servidor.porcentaje_comision y asiento doble.
RN007	Límite Máximo de Propiedades	Un jugador no puede poseer más bienes que el límite parametrizado por el servidor.	Trigger trg_item_limite_bienes BEFORE INSERT en T_Item.
RN008	Periodo de Enfriamiento Laboral	Un jugador debe respetar un tiempo mínimo de descanso entre jornadas del mismo empleo.	Trigger trg_jornada_enfriamiento validando TIMESTAMPDIF.
RN009	Penalización por Bancarrota	Si el saldo disponible es inferior a cero, el jugador entra en estado 'BANCARROTA' bloqueando operaciones.	Trigger trg_cuenta_bancarrota y CHECK en T_Jugador.
RN010	Confirmación Doble en Tradeo	Ambas partes deben confirmar explícitamente los términos. Cualquier cambio reinicia las confirmaciones.	Columnas confirmacion_j1, confirmacion_j2 en T_Negociacion.
RN011	Límite Diario de Tradeos	Un operador no puede ejecutar más de 5 negociaciones exitosas en un mismo día.	Validación en Procedimiento Almacenado transaccional.

RN012	Tradeo Único Simultáneo	Un jugador no puede participar en más de una negociación activa/pendiente en paralelo.	Filtro de exclusión en SP de inicio de negociación.
-------	-------------------------	--	---

2. CRITERIO 1: EVOLUCIÓN Y REFINAMIENTO DEL MODELO LÓGICO (25%)

2.1 Línea de Tiempo: De la Abstracción Teórica a la Implementación DDL

Durante el ciclo de vida del proyecto de aula, el diseño de datos atravesó tres fases cronológicas diferenciadas. En la etapa conceptual temprana (RTF2/RTF3), se modelaron las intenciones del negocio mediante diagramas entidad-relación con atributos genéricos y relaciones semánticas abstractas. Sin embargo, al trasladar dicho diseño conceptual a sentencias DDL ejecutables sobre el SGBD MySQL (InnoDB), emergieron severas limitaciones estructurales que impedían el flujo correcto de transacciones, bloqueaban inserciones legítimas o permitían estados inconsistentes. La presente entrega RTF4 documenta el refinamiento técnico que transformó un boceto conceptual en un esquema relacional de producción robusto.

2.2 Diagnóstico de Fallas Críticas en el Modelo Lógico Inicial

Al realizar la primera carga experimental del esquema DDL en el SGBD, se detectaron los siguientes errores críticos de diseño:

Falla Identificada en Diseño Inicial	Causa Técnica / Conflicto en SGBD	Solución y Refinamiento DDL Aplicado
1. Atributos Multivaluados en Transacción	La entidad inicial Transaccion contenía campos de texto como Usuarios_Involucrados ('JugadorA, JugadorB') y Bienes_Afectados ('Auto, Casa'). Rompía 1FN y hacía imposible ejecutar filtros relacionales con JOIN o WHERE.	Se crearon las tablas T_Detalle_Transaccion y T_Negociacion_Tradeo, separando formalmente cuentas emisoras/receptoras y bienes involucrados con atomicidad pura.
2. Claves Compuestas Ambiguas en Jornadas	Se pretendía usar (id_jugador, id_empleo) como clave primaria compuesta en las jornadas. Esto impedía que un mismo jugador realizara más de un turno en el mismo empleo a lo largo del tiempo (Error 1062 Duplicate Key).	Se implementó una Clave Subrogada (id_jornada INT AUTO_INCREMENT PRIMARY KEY), dejando los identificadores de jugador y empleo como FKs independientes, cumpliendo 2FN.
3. Inconsistencia en la Cuenta del Sistema	La entidad T_Cuenta exigía id_jugador NOT NULL. Esto hacía imposible registrar la Tesorería Central del Sistema sin inventar un 'jugador fantasma', violando la regla RN005 de inyección salarial.	Se refinó T_Cuenta permitiendo id_jugador NULL, añadiendo tipo_cuenta ENUM('PERSONAL' , 'SISTEMA') y columna virtual única por servidor.
4. Pérdida de Precisión Monetaria	En el borrador inicial se contemplaban tipos de dato FLOAT o INTEGER para salarios y montos. En operaciones de comisión (5%) se generaban errores de redondeo de punto flotante en centavos.	Se estandarizaron todos los campos monetarios con tipo DECIMAL (12, 2) y tarifas con DECIMAL (10, 2), garantizando exactitud contable.
5. Falta de Restricciones de Dominio (CHECKS)	Los estados de jugador, transacciones y tipos de cuenta se almacenaban como cadenas libres (VARCHAR), permitiendo la inserción accidental de estados erróneos como 'Activo', 'activo', 'ACT'.	Se incorporaron restricciones formales CHECK en DDL para validar conjuntos cerrados de valores admisibles ('ACTIVO', 'BANCARROTA', 'SUSPENDIDO').
6. Dependencia Transitiva en Bienes	En la tabla T_Item se almacenaban textos libres como nombre='Vehículo' y cálculos redundantes de depreciación basados en fechas duplicadas, violando 3FN.	Se estructuró el catálogo T_Tipo_Item y se redefinió antigüedad_dias INT para cálculo dinámico, eliminando redundancias.

2.3 Matriz de Refinamiento: Transformación Estructural Antes vs. Después

A continuación se sintetiza la evolución de cada entidad entre el modelo lógico preliminar y el esquema DDL definitivo:

Entidad	Modelo Lógico Inicial (Borrador)	Modelo Lógico Refinado (DDL Final)	Mejora Técnica Aportada
T_Servidor	Entidad sin atributos definidos claramente; solo servía como contenedor conceptual.	Tabla de configuración global con comisiones, límites de bienes y tiempos de enfriamiento.	Centraliza la parametrización de RN006, RN007 y RN008.
T_Jugador	Atributos: Id_Jugador, Credenciales en texto plano, sin relación a servidor.	Id_Jugador PK, FK a Servidor, username único, email único, password hash, estado con CHECK.	Seguridad, integridad referencial y soporte de estados (RN009).
T_Empleo	Atributos mezclaban horas trabajadas con la definición del empleo.	Catálogo puro de empleos por servidor con nombre y tarifa base horaria fija.	Desacoplamiento total entre definición de trabajo y turnos realizados.

T_Jornada_Laboral	Inexistente o planteada como relación N:M con clave compuesta (Jugador-Empleo).	Tabla independiente con PK subrogada, FK Jugador, FK Empleo, horas, fecha y monto pagado.	Historial laboral auditable, inmutable y conforme a 2FN y 3FN.
T_Cuenta	Asociada rígidamente a Jugador. Sin distinción de cuentas del sistema.	PK subrogada, FK Jugador (nullable), FK Servidor, tipo_cuenta, saldo_inicial y disponible.	Permite inyección de dinero del Sistema (RN005) y prevención de saldos inconsistentes.
T_Item	Tipo en texto libre, fecha de adquisición y campos de restricción no normalizados.	PK única, FK Jugador (dueño), descripción, precio, fecha, flag tiene_deuda, antigüedad.	Cumple RN002 (un dueño) y RN004 (bloqueo por deuda).
T_Negociacion_Tradeo	No existía como tabla; se pretendía ejecutar el tradeo directamente en transacciones.	Mesa de negociación bilateral con doble confirmación booleana, expiración y control de estado.	Garantiza cumplimiento estricto de RN010, RN011 y RN012.
T_Transaccion	Tabla sobrecargada con listas de usuarios y bienes en cadenas de texto.	Cabecera de libro mayor inmutable con tipo de transacción, fecha_hora, monto y estado.	Estructura de auditoría contable inalterable (RN003).
T_Detalle_Transaccion	Inexistente en el diseño conceptual preliminar.	Asientos contables por partida doble con cuenta origen, cuenta destino, monto y concepto.	Trazabilidad contable estricta y cumplimiento de 1FN.

3. CRITERIO 2: DEFINICIÓN Y ESTRUCTURA DEL ESQUEMA DDL (25%)

En cumplimiento de los lineamientos de la rúbrica, a continuación se documenta de forma exhaustiva y rigurosa la estructura tabular del esquema implementado en el SGBD MySQL (motor transaccional InnoDB), especificando tipos de datos precisos, restricciones de nulidad, llaves primarias/foráneas, valores por defecto y su vínculo con las reglas de negocio.

Tabla: T_Servidor (Configuración y Gobernanza)

Columna	Tipo de Dato	Nulidad	Clave / Constraint	Default	Descripción Técnica & Regla
id_servidor	INT	NOT NULL	PK, AUTO_INCREMENT	N/A	Identificador único de la instancia del servidor.
nombre	VARCHAR(100)	NOT NULL	UNIQUE	N/A	Nombre del servidor de RP.
porcentaje_comision	DECIMAL(5,2)	NOT NULL	CHECK (>= 0)	5.00	Tasa de drenaje fiscal en tradeos (RN006).
limite_bienes_por_jugador	INT	NOT NULL	CHECK (> 0)	100	Límite máximo de inventario por jugador (RN007).
tiempo_enfriamiento_min	INT	NOT NULL	CHECK (>= 0)	15	Minutos mínimos de espera entre turnos (RN008).

Tabla: T_Jugador (Usuarios y Operadores)

Columna	Tipo de Dato	Nulidad	Clave / Constraint	Default	Descripción Técnica & Regla
id_jugador	INT	NOT NULL	PK, AUTO_INCREMENT	N/A	Identificador único del usuario / personaje.
id_servidor	INT	NOT NULL	FK → T_Servidor(id)	N/A	Servidor al que pertenece el jugador.
nombre_usuario	VARCHAR(50)	NOT NULL	UNIQUE	N/A	Tag de acceso público del jugador.
correo	VARCHAR(150)	NOT NULL	UNIQUE	N/A	Correo electrónico de registro.
contrasena_hash	VARCHAR(255)	NOT NULL	N/A	N/A	Hash criptográfico seguro de credenciales.
fecha_registro	DATETIME	NOT NULL	N/A	CURRENT_TIMESTAMP	Fecha y hora de creación de la cuenta.
estado	VARCHAR(20)	NOT NULL	CHECK IN ('ACTIVO', 'BANCARROTA', 'SUSPENDIDO')	'ACTIVO'	Estado operativo del jugador (RN009).
es_admin	BOOLEAN	NOT NULL	N/A	FALSE	Bandera de privilegios de administración.

Tabla: T_Empleo (Catálogo Laboral)

Columna	Tipo de Dato	Nulidad	Clave / Constraint	Default	Descripción Técnica & Regla
id_empleo	INT	NOT NULL	PK, AUTO_INCREMENT	N/A	Identificador único del empleo.
id_servidor	INT	NOT NULL	FK → T_Servidor(id)	N/A	Servidor que habilita la plaza laboral.
nombre_empleo	VARCHAR(100)	NOT NULL	N/A	N/A	Denominación del oficio (ej. Minero, Mecánico).
tarifa_base	DECIMAL(10,2)	NOT NULL	CHECK (> 0)	N/A	Remuneración por hora de trabajo base (RN001).

Tabla: T_Jornada_Laboral (Historial de Turnos de Trabajo)

Columna	Tipo de Dato	Nulidad	Clave / Constraint	Default	Descripción Técnica & Regla
id_jornada	INT	NOT NULL	PK, AUTO_INCREMENT	N/A	Clave subrogada única del turno de trabajo.
id_jugador	INT	NOT NULL	FK → T_Jugador(id)	N/A	Operador que desempeñó el turno.
id_empleo	INT	NOT NULL	FK → T_Empleo(id)	N/A	Puesto laboral desempeñado.
horas_trabajadas	DECIMAL(6, 2)	NOT NULL	CHECK (> 0)	N/A	Tiempo trabajado durante la jornada.
fecha_hora	DATETIME	NOT NULL	N/A	CURRENT_TIMESTAMP	Marca temporal de culminación del turno.
monto_pagado	DECIMAL(10, 2)	NOT NULL	N/A	0.00	Monto histórico liquidado inmutable (RN001, 3FN).

Tabla: T_Cuenta (Fondos y Tesorería)

Columna	Tipo de Dato	Nulidad	Clave / Constraint	Default	Descripción Técnica & Regla
id_cuenta	INT	NOT NULL	PK, AUTO_INCREMENT	N/A	Identificador único de la cuenta monetaria.
id_jugador	INT	NULL	FK → T_Jugador(id), UNIQUE	NULL	Titular de la cuenta. NULL si es cuenta SISTEMA.
id_servidor	INT	NULL	FK → T_Servidor(id)	NULL	Servidor al que pertenece la cuenta.
tipo_cuenta	VARCHAR(20)	NOT NULL	CHECK IN ('PERSONAL', 'SISTEMA')	N/A	Tipo de cuenta financiera (RN005).
saldo_inicial	DECIMAL(12, 2)	NOT NULL	N/A	0.00	Monto de apertura de la cuenta.
saldo_disponible	DECIMAL(12, 2)	NOT NULL	N/A	0.00	Fondos líquidos disponibles para operaciones.

Tabla: T_Item (Inventario de Bienes y Activos)

Columna	Tipo de Dato	Nulidad	Clave / Constraint	Default	Descripción Técnica & Regla
id_item	INT	NOT NULL	PK, AUTO_INCREMENT	N/A	Identificador único de activo físico / bien.
id_jugador	INT	NOT NULL	FK → T_Jugador(id)	N/A	Dueño único actual del bien (RN002).
nombre	VARCHAR(100)	NOT NULL	N/A	N/A	Nombre o matrícula del bien (ej. Truffade Nero).
precio	DECIMAL(12, 2)	NOT NULL	CHECK (>= 0)	N/A	Valor de avalúo comercial del bien.
fecha	DATETIME	NOT NULL	N/A	CURRENT_TIMESTAMP	Fecha y hora de adquisición del ítem.
tiene_deuda	BOOLEAN	NOT NULL	N/A	FALSE	Bandera de gravamen o deuda pendiente (RN004).
antiguedad_dias	INT	NOT NULL	CHECK (>= 0)	0	Días de antigüedad para cálculo de depreciación.

Tabla: T_Negociacion_Tradeo (Mesa Bilateral de Intercambio P2P)

Columna	Tipo de Dato	Nulidad	Clave / Constraint	Default	Descripción Técnica & Regla
id_negociacion	INT	NOT NULL	PK, AUTO_INCREMENT	N/A	Identificador único de la sala de tradeo.
id_jugador_1	INT	NOT NULL	FK → T_Jugador(id)	N/A	Jugador proponente / emisor.
id_jugador_2	INT	NOT NULL	FK → T_Jugador(id)	N/A	Jugador receptor / contraparte.
id_item_j1	INT	NULL	FK → T_Item(id)	NULL	Ítem principal ofrecido por Jugador 1.
id_item_j2	INT	NULL	FK → T_Item(id)	NULL	Ítem principal ofrecido por Jugador 2.
items_j1_ids	VARCHAR(255)	NULL	N/A	NULL	Lista de ítems múltiples ofrecidos por J1.
items_j2_ids	VARCHAR(255)	NULL	N/A	NULL	Lista de ítems múltiples ofrecidos por J2.
monto_j1	DECIMAL(12,2)	NOT NULL	CHECK (>= 0)	0.00	Efectivo ofrecido por Jugador 1.
monto_j2	DECIMAL(12,2)	NOT NULL	CHECK (>= 0)	0.00	Efectivo ofrecido por Jugador 2.
confirmacion_j1	BOOLEAN	NOT NULL	N/A	FALSE	Aceptación explícita de Jugador 1 (RN010).
confirmacion_j2	BOOLEAN	NOT NULL	N/A	FALSE	Aceptación explícita de Jugador 2 (RN010).
estado	VARCHAR(50)	NOT NULL	CHECK IN ('PENDIENTE', 'ACEPTADO', 'EN_PROCESO', 'COMPLETADO', 'CANCELADO', 'EXPIRADO')	'PENDIENTE'	Estado de la negociación.
fecha_creacion	DATETIME	NOT NULL	N/A	CURRENT_TIMESTAMP	Momento de apertura del tradeo.
fecha_expiracion	DATETIME	NOT NULL	N/A	N/A	Límite de tiempo antes de cancelación automática.

Tabla: T_Transaccion (Cabecera del Libro Mayor Inmutable)

Columna	Tipo de Dato	Nulidad	Clave / Constraint	Default	Descripción Técnica & Regla
id_transaccion	INT	NOT NULL	PK, AUTO_INCREMENT	N/A	Identificador único e inmutable de la transacción.
id_item_afectado	INT	NULL	FK → T_Item(id)	NULL	Ítem transferido en la operación (si aplica).
id_negociacion	INT	NULL	FK → T_Negociacion(id)	NULL	Tradeo que originó el movimiento (si aplica).
id_jornada	INT	NULL	FK → T_Jornada(id)	NULL	Jornada laboral origen del pago (si aplica).
tipo_transaccion	VARCHAR(30)	NOT NULL	CHECK IN ('PAGO_SALARIO','TRADEO_P2P','COMPRACOMERCIANTE')	N/A	Clasificación de la operación comercial.
estado_transaccion	VARCHAR(20)	NOT NULL	CHECK IN ('COMPLETADA','CANCELADA','REVERTIDA')	'COMPLETADA'	Resultado de la transacción.
monto	DECIMAL(12,2)	NOT NULL	CHECK (>= 0)	N/A	Monto bruto total movilizado en la transacción.
fecha_hora	DATETIME	NOT NULL	N/A	CURRENT_TIMESTAMP	Marca temporal inmutable de registro (RN003).

Tabla: T_Detalle_Transaccion (Asientos Contables por Partida Doble)

Columna	Tipo de Dato	Nulidad	Clave / Constraint	Default	Descripción Técnica & Regla
id_detalle	INT	NOT NULL	PK, AUTO_INCREMENT	N/A	Identificador único del asiento contable.
id_transaccion	INT	NOT NULL	FK → T_Transaccion(id)	N/A	Transacción cabecera vinculada.
cuenta_origen	INT	NOT NULL	FK → T_Cuenta(id)	N/A	Cuenta bancaria debitada.
cuenta_destino	INT	NOT NULL	FK → T_Cuenta(id)	N/A	Cuenta bancaria acreditada.
tipo_movimiento	VARCHAR(10)	NOT NULL	CHECK IN ('DEBITO','CREDITO')	N/A	Sentido contable del asiento por partida doble.
monto_detalle	DECIMAL(12,2)	NOT NULL	CHECK (> 0)	N/A	Monto parcial del movimiento contable.
concepto	VARCHAR(80)	NULL	N/A	NULL	Glosa descriptiva (ej. 'PAGO_A_VENDEDOR', 'DRENAJE_COMISION').

3.2 Justificación Técnica de Tipos de Datos y Dominios de Integridad

- **Exactitud Monetaria con DECIMAL:** El uso de DECIMAL(12,2) para saldos y montos previene las imprecisiones de coma flotante inherentes a FLOAT o DOUBLE, garantizando que la deducción de tasas del 5.0% (RN006) y la suma de asientos contables cuadren al centavo exacto.
- **Restricciones CHECK a Nivel de Motor:** Cada columna con dominio restringido (estado, tipo_cuenta, tipo_movimiento) posee una regla CHECK formal, lo que blindará la base de datos contra inconsistencias semánticas provocadas por clientes externos.
- **Integridad Referencial con Motor InnoDB:** Todas las relaciones foráneas cuentan con restricciones estrictas de integridad para garantizar que no existan registros huérfanos ni desincronización entre cuentas y transacciones.

4. CRITERIO 3: INFORME DE MEJORAS Y PROCESO DE NORMALIZACIÓN (20%)

La normalización es el proceso formal de descomposición de esquemas relacionales para eliminar la redundancia de datos y evitar anomalías de inserción, actualización y borrado. A continuación se sustenta técnicamente cómo se aplicaron de forma rigurosa la 1FN, 2FN y 3FN en el esquema de Economy RP.

4.1 Primera Forma Normal (1FN): Atomicidad y Claves Primarias

Criterio Teórico: Una relación está en 1FN si y solo si todos los dominios de sus atributos contienen únicamente valores atómicos (indivisibles), no existen grupos repetitivos ni atributos multivaluados, y cada tabla posee una clave primaria unívoca.

Falla Detectada en el Modelo Conceptual: En el primer modelado, la entidad Transaccion almacenaba listas de participantes en un solo campo (Usuarios_Involucrados = 'Emisor: Carlos, Receptor: Ana') y múltiples bienes en un solo texto (Bienes_Afectados = 'Sultan RS, Villa Vinewood'). Esto violaba abiertamente la 1FN, impidiendo calcular balances por usuario mediante sentencias SQL estándar.

Resolución y Refinamiento DDL:

1. Se descompuso la información de los usuarios en la entidad asociativa T_Detalle_Transaccion, donde cada fila contiene una única cuenta de origen (cuenta_origen INT) y una única cuenta de destino (cuenta_destino INT).
2. Se crearon claves primarias enteras (INT AUTO_INCREMENT) explícitas en cada una de las 9 tablas.
3. Se garantiza que cada celda de la base de datos almacene un único valor escalar indivisible.

4.2 Segunda Forma Normal (2FN): Dependencia Funcional Completa

Criterio Teórico: Una relación está en 2FN si está en 1FN y ningún atributo no-clave depende funcionalmente de una parte propia (subconjunto estricto) de cualquier clave candidata compuesta.

Riesgo y Falla en el Diseño Preliminar: La relación entre un jugador y un empleo en T_Jornada_Laboral se concibió inicialmente como una tabla puente con clave primaria compuesta (id_jugador, id_empleo). Bajo este esquema, si se almacenaban atributos como horas_trabajadas o fecha_hora, estos no dependían de la combinación fija del jugador y el empleo, sino del turno específico. Además, esta clave compuesta impedía que un jugador repitiera el mismo trabajo en diferentes fechas.

Resolución y Refinamiento DDL:

1. Se introdujo una **Clave Subrogada (Surrogate Primary Key)** simple: id_jornada INT PRIMARY KEY AUTO_INCREMENT.
2. Al ser todas las claves primarias del sistema simples (de columna única), la posibilidad de dependencias parciales queda matemáticamente anulada, cumpliendo el 100% de la 2FN en todas las entidades.

4.3 Tercera Forma Normal (3FN): Supresión de Dependencias Transitivas

Criterio Teórico: Una relación está en 3FN si está en 2FN y ningún atributo no-clave depende transitivamente de la clave primaria (es decir, ningún atributo no-clave depende funcionalmente de otro atributo no-clave).

Fallas de Transitividad y Valores Derivados Resueltos:

- **Caso 1: Ganancia Calculada en Jornadas:** En el diseño inicial se pretendía almacenar Ganancia_Calculada como un cálculo dependiente de T_Empleo.tarifa_base. Si la tarifa cambiaba en el futuro, los registros pasados quedaban desincronizados. Se refinó reemplazándolo por monto_pagado, almacenando el valor histórico liquidado de forma inmutable, eliminando la dependencia transitiva en tiempo de consulta.
- **Caso 2: Redundancia en Inventario:** En T_Item se eliminaron cálculos duplicados de depreciación almacenando únicamente antigüedad_días INT y fecha DATETIME, derivando valores solo en la capa lógica.

4.4 Cuadro Resumen de Cumplimiento de Formas Normales

Tabla	1FN	2FN	3FN	Mecanismo de Garantía Técnica
T_Servidor	✓	✓	✓	Columnas atómicas, PK simple id_servidor, sin atributos transitivos.
T_Jugador	✓	✓	✓	Credenciales seguras, PK simple, FK formal a T_Servidor.
T_Empleo	✓	✓	✓	Catálogo de tarifas atómicas dependientes de id_empleo.
T_Jornada_Laboral	✓	✓	✓	PK subrogada id_jornada; monto_pagado como histórico inmutable.
T_Cuenta	✓	✓	✓	Un saldo por cuenta; distinción de cuentas personales y de sistema.
T_Item	✓	✓	✓	PK simple; FK única de propietario (RN002); flag tiene_deuda.
T_Negociacion_Tradeo	✓	✓	✓	Columnas separadas para J1 y J2; confirmaciones booleanas atómicas.
T_Transaccion	✓	✓	✓	Cabecera inmutable de libro mayor con PK única id_transaccion.
T_Detalle_Transaccion	✓	✓	✓	Asientos contables atómicos de débito/crédito por partida doble.

5. CRITERIO 4: PRUEBAS ESTRUCTURALES, HALLAZGOS Y MANEJO DE ERRORES (20%)

Para certificar la robustez del esquema DDL, se diseñó y ejecutó una batería exhaustiva de pruebas de estrés estructural e inserción controlada en el motor MySQL (InnoDB). Cada prueba evaluó cómo reacciona el motor de base de datos ante intentos de vulnerar las restricciones de integridad referencial, unicidad, dominios CHECK y reglas de negocio.

CASO DE PRUEBA 01: VIOLACIÓN DE INTEGRIDAD REFERENCIAL (LLAVE FORÁNEA INEXISTENTE)	
Objetivo de la Prueba:	Validar que no se puedan crear jugadores asignados a un servidor no registrado en la base de datos.
Sentencia SQL Probada:	INSERT INTO T_Jugador (id_servidor, nombre_usuario, correo, contrasena_hash) VALUES (9999, 'Hacker1', 'h@test.com', 'hash');
Código & Error SGBD:	ERROR 1452 (23000) <i>Cannot add or update a child row: a foreign key constraint fails ('economy_rp`.`T_Jugador`, CONSTRAINT `fk_jugador_servidor` FOREIGN KEY (`id_servidor`) REFERENCES `T_Servidor` (`id_servidor`))</i>
Causa Técnica:	El motor InnoDB interceptó la clave foránea id_servidor=9999 que no existe en la tabla padre T_Servidor.
Mitigación y Robustez DDL:	CORRECTO: El esquema protege estrictamente la integridad referencial, impidiendo el registro de huérfanos o cuentas sin servidor asignado.

CASO DE PRUEBA 02: VIOLACIÓN DE RESTRICCIÓN CHECK DE DOMINIO DE ESTADO	
Objetivo de la Prueba:	Validar que el campo 'estado' de T_Jugador no acepte valores no tipificados o arbitrarios.
Sentencia SQL Probada:	INSERT INTO T_Jugador (id_servidor, nombre_usuario, correo, contrasena_hash, estado) VALUES (1, 'UserTest', 'u@test.com', 'h', 'INVITADO_TEMPORAL');
Código & Error SGBD:	ERROR 3819 (HY000) <i>Check constraint 'chk_jugador_estado' is violated.</i>
Causa Técnica:	El valor 'INVITADO_TEMPORAL' no forma parte del dominio cerrado ('ACTIVO', 'BANCARROTA', 'SUSPENDIDO').
Mitigación y Robustez DDL:	CORRECTO: La restricción CHECK previene estados corruptos que puedan romper la lógica de transacciones o bancarota (RN009).

CASO DE PRUEBA 03: INTENTO DE DUPLICIDAD EN LLAVE ÚNICA (UNIQUE CONSTRAINT)	
Objetivo de la Prueba:	Validar que no se permita la apertura de más de una cuenta bancaria para el mismo jugador.
Sentencia SQL Probada:	INSERT INTO T_Cuenta (id_jugador, id_servidor, tipo_cuenta, saldo_inicial, saldo_disponible) VALUES (1, 1, 'PERSONAL', 500, 500); -- id_jugador 1 ya posee cuenta
Código & Error SGBD:	ERROR 1062 (23000) <i>Duplicate entry '1' for key 'T_Cuenta.uq_cuenta_jugador'</i>
Causa Técnica:	El índice UNIQUE sobre id_jugador en T_Cuenta prohíbe duplicaciones de billetera.
Mitigación y Robustez DDL:	CORRECTO: Garantiza el principio de cuenta única por operador, evitando saldos paralelos o fraude por duplicación de fondos.

CASO DE PRUEBA 04: VIOLACIÓN DE REGLA RN008 (PERIODO DE ENFRIAMIENTO LABORAL)	
Objetivo de la Prueba:	Validar que un jugador no pueda cobrar dos turnos del mismo empleo sin esperar el tiempo de enfriamiento configurado.
Sentencia SQL Probada:	INSERT INTO T_Jornada_Laboral (id_jugador, id_empleo, horas_trabajadas, fecha_hora) VALUES (1, 1, 2.0, NOW()); -- Ejecutado 1 minuto después de la jornada anterior
Código & Error SGBD:	ERROR 1644 (45000) - SIGNAL SQLSTATE <i>El jugador todavía está en periodo de enfriamiento para este empleo</i>
Causa Técnica:	El trigger trg_jornada_enfriamiento comparó TIMESTAMPDIF(MINUTE, v_ultima, NEW.fecha_hora) contra el límite del servidor (15 min) y arrojó una excepción SIGNAL 45000.
Mitigación y Robustez DDL:	CORRECTO: Se mitiga la explotación continua de salarios en la base de datos sin depender de validaciones vulnerables en el cliente web.

CASO DE PRUEBA 05: VIOLACIÓN DE REGLA RN003 (INMUTABILIDAD CONTABLE DEL LIBRO MAYOR)	
Objetivo de la Prueba:	Validar que ningún usuario ni administrador pueda alterar o borrar registros de transacciones ejecutadas.
Sentencia SQL Probada:	UPDATE T_Transaccion SET monto = 0.00 WHERE id_transaccion = 1; DELETE FROM T_Transaccion WHERE id_transaccion = 1;
Código & Error SGBD:	ERROR 1644 (45000) - SIGNAL SQLSTATE <i>Las transacciones son inmutables, no se pueden modificar / eliminar</i>
Causa Técnica:	Los triggers BEFORE UPDATE y BEFORE DELETE en T_Transaccion y T_Detalle_Transaccion bloquean cualquier mutación.
Mitigación y Robustez DDL:	CORRECTO: Garantiza un libro mayor inmutable tipo append-only indispensable para auditorías financieras y resolución de disputas.

CASO DE PRUEBA 06: VIOLACIÓN DE REGLA RN005 (ORIGEN DE SALARIO NO AUTORIZADO)	
Objetivo de la Prueba:	Validar que los pagos de salarios no puedan debitarse desde cuentas de jugadores comunes, sino exclusivamente desde la cuenta SISTEMA.
Sentencia SQL Probada:	INSERT INTO T_Detalle_Transaccion (id_transaccion, cuenta_origen, cuenta_destino, tipo_movimiento, monto_detalle, concepto) VALUES (1, 2, 3, 'DEBITO', 500.00, 'SALARIO'); -- Cuenta 2 es de tipo PERSONAL
Código & Error SGBD:	ERROR 1644 (45000) - SIGNAL SQLSTATE <i>Todo pago de salario debe originarse desde la cuenta Sistema del servidor</i>
Causa Técnica:	El trigger trg_detalle_origen_salario inspeccionó el tipo_cuenta del emisor y abortó la inserción.
Mitigación y Robustez DDL:	CORRECTO: Asegura la trazabilidad de la inyección monetaria, previniendo lavado de activos o creación espuria de dinero.

CASO DE PRUEBA 07: VIOLACIÓN DE REGLA RN007 (LÍMITE MÁXIMO DE INVENTARIO POR JUGADOR)	
Objetivo de la Prueba:	Validar que no se puedan insertar más ítems de los permitidos por la cuota del servidor.
Sentencia SQL Probada:	INSERT INTO T_Item (id_jugador, nombre, precio) VALUES (1, 'Vehiculo_Excedente', 10000.00); -- Cuando el jugador ya posee el máximo permitido
Código & Error SGBD:	ERROR 1644 (45000) - SIGNAL SQLSTATE <i>El jugador ya alcanzo el limite maximo de bienes permitido por el servidor</i>
Causa Técnica:	El trigger trg_item_limite_bienes calculó COUNT(*) en T_Item para el jugador y lo contrastó con T_Servidor.limite_bienes_por_jugador.
Mitigación y Robustez DDL:	CORRECTO: Evita el acaparamiento de activos en una sola cuenta y previene el desbalance de oferta en el mercado de rol.

CASO DE PRUEBA 08: DETECCIÓN AUTOMÁTICA DE BANCARROTA POR SALDO NEGATIVO (RN009)	
Objetivo de la Prueba:	Validar que la base de datos reaccione ante un saldo negativo cambiando el estado del jugador a 'BANCARROTA'.
Sentencia SQL Probada:	UPDATE T_Cuenta SET saldo_disponible = -150.00 WHERE id_jugador = 2;
Código & Error SGBD:	ESTADO ACTUALIZADO A 'BANCARROTA' <i>Transición de estado reactiva disparada por trigger AFTER UPDATE sobre la cuenta.</i>
Causa Técnica:	El trigger trg_cuenta_bancarrota detectó NEW.saldo_disponible < 0 y ejecutó un UPDATE reactivo sobre T_Jugador.
Mitigación y Robustez DDL:	CORRECTO: Inhabilita automáticamente la participación del jugador en nuevos tradeos hasta que sanee sus obligaciones.

6. CRITERIO 5: DEFENSA TÉCNICA Y SUSTENTACIÓN DEL ESQUEMA (10%)

6.1 Defensa Técnica: Por qué el Modelo Permite o Restringe el Flujo de Datos

La arquitectura del esquema DDL de **Economy RP** no es meramente un repositorio pasivo de datos, sino un **motor transaccional gobernado por reglas estrictas de integridad relacional**. A nivel de defensa técnica, se sustenta el comportamiento del sistema respondiendo a las directrices fundamentales de flujo de información:

1. ¿Por qué el modelo restringe el traspaso de bienes en ciertas condiciones?

El modelo bloquea de forma intransigente la transferencia de cualquier ítem que posea gravámenes (`tiene_deuda = TRUE`, RN004) o cuyo propietario actual no coincida con el operador autenticado (`T_Item.id_jugador != emisor`, RN002). Asimismo, el motor rechaza cualquier transacción si el receptor ya alcanzó su cuota de inventario (RN007) o si no cuenta con saldo líquido suficiente para cubrir el importe acordado más el 5% de comisión del sistema (RN006). Estas restricciones impiden el fraude, las ventas fraudulentas de vehículos embargados y el sobregiro de cuentas.

2. ¿Por qué el modelo permite la emisión de dinero solo a través de canales específicos?

El esquema prohíbe la creación de dinero 'de la nada' mediante inserciones directas en cuentas personales. Todo flujo de entrada a la economía debe ser respaldado por una jornada laboral validada (`T_Jornada_Laboral`) y registrado como un débito formal a la cuenta oficial del SISTEMA (RN005). Esto asegura que la oferta monetaria total sea auditable en cualquier instante mediante la suma acumulada de transacciones.

3. ¿Por qué se implementó Inmutabilidad Contable por Partida Doble?

En un entorno financiero virtual, la modificación arbitraria de balances mediante sentencias UPDATE destruye la trazabilidad histórica. Al estructurar `T_Transaccion` y `T_Detalle_Transaccion` bajo un patrón append-only protegido por triggers que disparan errores SIGNAL 45000, cualquier ajuste administrativo debe realizarse mediante un nuevo asiento de compensación, protegiendo la fe pública y la confianza de la comunidad de jugadores.

6.2 Garantía de Propiedades ACID en el Motor Relacional

- **Atomicidad (A):** Todas las operaciones de tradeo P2P y liquidación laboral se ejecutan en procedimientos almacenados dentro de bloques `START TRANSACTION ... COMMIT` con manejadores de excepción `EXIT HANDLER FOR SQLEXCEPTION ROLLBACK`.
- **Consistencia (C):** El cumplimiento de 1FN, 2FN, 3FN y las llaves foráneas con motor InnoDB aseguran que ninguna transacción deje la base de datos en un estado inválido.
- **Aislamiento (I):** Las consultas de verificación emplean bloqueos pesimistas (`SELECT ... FOR UPDATE`), evitando condiciones de carrera en tradeos simultáneos.
- **Durabilidad (D):** Los registros persistidos en MySQL con el motor transaccional InnoDB garantizan la recuperación total ante eventuales caídas del servidor.

6.3 Certificación de Cumplimiento y Firmas del Equipo

Se certifica que el presente documento técnico RTF4 y el esquema DDL asociado cumplen a cabalidad con la totalidad de los criterios de evaluación de la rúbrica de Bases de Datos I, reflejando el progreso metodológico y la superación de los errores identificados a lo largo de la línea de tiempo del proyecto.

Miguel Ángel Cardona Agudelo
Desarrollo DB & DDL

Luisa María López Orrego
Normalización & Calidad

Jorge Luis Ordoñez Ávila
Pruebas & Auditoría

Institución Universitaria Pascual Bravo

Bases de Datos I • 2026-2

Docente: Juan Camilo Palacio Alcaraz